

A Comparative Study of Classical vs Quantum Query Complexity using the Deutsch–Jozsa Algorithm with Qiskit Implementation

by:

Ashwanth. B

[Grade 11]

Abstract

This paper explores the difference between classical and quantum query complexity using the Deutsch–Jozsa algorithm. The objective is to show how quantum computers can reduce the number of evaluations compared to classical computations. The algorithm is implemented using Qiskit on a quantum simulator. The results show that while classical algorithms may require multiple queries, the quantum approach determines the function's nature with a single query, illustrating a clear quantum advantage.

1. Introduction

In classical computing, the basic unit of data is the *bit*, which can exist in one of two states: 0 or 1.

In contrast, quantum computing uses the *quantum bit*, or *qubit*, as its fundamental unit of information. Unlike classical bits, qubits can exist in a superposition of states, allowing them to represent both 0 and 1 simultaneously.

Quantum computing takes a completely different approach — instead of forcing information into rigid 0s and 1s, it lets the underlying strangeness of quantum mechanics do the computational heavy lifting.

This paper compares classical and quantum approaches to show how the Deutsch–Jozsa algorithm can determine the nature of a function using fewer function evaluations than a classical method.

To understand further, it helps to first get a feel for the building blocks — qubits, superposition, and the gates that manipulate them.

2. Background

2.1 Qubits

A classical bit is decisive — it's either 0 or 1, nothing in between. A qubit refuses to commit. Until you actually look at it, it can exist as some combination of both 0 and 1 simultaneously, a behaviour that has no real analogy in everyday experience.

In other words, a qubit can exist in state 0, state 1, or a superposition of both states.

Qubits can be physically represented using systems such as the spin of an electron or the polarisation of a photon.

2.2 Superposition

Before measurement, a qubit in superposition genuinely holds both possibilities at once — not just an unknown value waiting to be revealed.

Upon measurement, the qubit collapses to either 0 or 1 with certain probabilities.

Mathematically, a qubit in superposition is represented as a linear combination of basis states:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Here, α and β are probability amplitudes. The probabilities of measuring the qubit in states 0 and 1 are given by $|\alpha|^2$ and $|\beta|^2$, respectively, with the condition:

$$|\alpha|^2 + |\beta|^2 = 1$$

A commonly used special case in quantum algorithms is:

$$\frac{1}{\sqrt{2}}(|0\rangle + (-1)^x|1\rangle)$$

where $x \in \{0,1\}$. This introduces a phase factor that plays an important role in quantum interference.

2.3 Quantum Gates

Where classical logic gates push bits from one state to another, quantum gates do something subtler — they rotate a qubit's state through a space of amplitudes, nudging probabilities without collapsing them.

Mathematically, quantum gates are represented by unitary matrices, which transform the state of a qubit while preserving total probability. When a quantum gate is applied, it changes the probability amplitudes of the qubit's state.

Quantum gates are reversible operations, meaning the original information can always be recovered after computation.

2.3.1 Hadamard Gate

The Hadamard gate creates superposition:

$$H|0\rangle = |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \qquad H|1\rangle = |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

2.3.2 X Gate

The X gate flips the state of a qubit. This gate corresponds to the NOT gate.

$$X|0\rangle = |1\rangle \qquad X|1\rangle = |0\rangle$$

2.3.3 CNOT Gate

The Controlled-NOT (CNOT) gate operates on two qubits: a *control qubit* and a *target qubit*. The target qubit is flipped only if the control qubit is 1.

Examples:

$$|00\rangle \rightarrow |00\rangle \qquad |01\rangle \rightarrow |01\rangle \qquad |10\rangle \rightarrow |11\rangle \qquad |11\rangle \rightarrow |10\rangle$$

2.4 The Problem:

The Deutsch algorithm addresses the following question:

Given a function that takes a single binary input and produces a binary output, how many queries are required to determine its nature?

There are four possible functions:

1. Identity: $f_0(x) = x$ $\begin{cases} f(0) = 0 \\ f(1) = 1 \end{cases}$
2. Negation: $f_1(x) = \neg x$ $\begin{cases} f(0) = 1 \\ f(1) = 0 \end{cases}$
3. Constant 0: $f_2(x) = 0$ $\begin{cases} f(0) = 0 \\ f(1) = 0 \end{cases}$
4. Constant 1: $f_3(x) = 1$ $\begin{cases} f(0) = 1 \\ f(1) = 1 \end{cases}$

A classical computer requires two queries to determine the function.

A quantum computer can determine *whether the function is constant or balanced* using only one query.

A function is called:

- Constant if it produces the same output for all inputs
- Balanced if it produces equal numbers of 0s and 1s

The Deutsch–Jozsa algorithm extends the original problem to functions that take n-bit binary strings as input instead of a single bit.

With the gates in hand, the Deutsch problem becomes not just solvable — it becomes a natural thing to try.

3. Methodology:

3.1 The Classical Approach:

In the classical approach, the function is evaluated by querying it with different inputs and observing the outputs. The goal is to determine whether the function is *constant* (outputs the same for all inputs) or *balanced* (outputs an equal number of 0s and 1s).

We can evaluate our given function at either 0 or 1. Suppose we evaluate it by plugging in 0; then there are two possible outcomes—either 0 or 1. If we get 0, all we know is $f(0)=0$. The function could be either f_0 or f_2 . Since one is constant and the other is balanced, we are forced to evaluate our function again with $f(1)$ to decide between them. Classically, to answer the question, we have to plug both 0 and 1 into the function. We need to make two function evaluations.

This may seem easy when $n=1$. But as n becomes bigger, the number of possible inputs increases as well.

For a function that takes an n-bit input, there are 2^n possible inputs. A classical computer must evaluate the function for multiple inputs to determine its nature. In the worst case, it may require up to:

$$2^{n-1} + 1$$

queries. This is because the first 2^{n-1} inputs (which are half of the inputs) may all produce the same output, and one additional query is needed to confirm whether the function changes.

In the worst case, this approach requires checking many inputs, demonstrating the inefficiency of classical computation for this problem.

This exponential blowup is exactly the wall the quantum approach walks straight through.

3.2 The Quantum Approach:

The Deutsch and Deutsch–Jozsa algorithms solve this problem using quantum mechanics. By using superposition, phase encoding, and interference, they determine whether a function is constant or balanced with only a *single oracle call*.

3.2.1 Phase Oracle:

Since any quantum oracle should be reversible, i.e., no information should be lost, the oracle will be used in the following way, where $i \in \{0,1,2,3\}$:

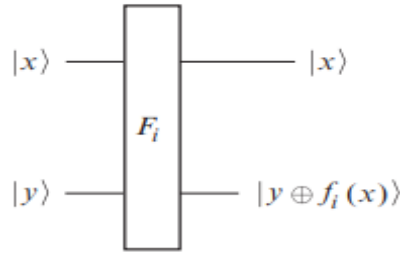


Figure 1: Reversible quantum circuit.

When the oracle is applied to the superposition state, the function's output is transferred to the phase of the quantum state. This effect is known as *phase kickback*.

Mathematically,

Apply Hadamard to 1 to obtain a superposition state " $|-\rangle$ " and initialise it along with $|x\rangle$.

$$\begin{aligned} |x\rangle|-\rangle &= |x\rangle\left(\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)\right) \\ &= \frac{1}{\sqrt{2}}(|x\rangle|0\rangle - |x\rangle|1\rangle) \end{aligned}$$

Applying Oracle,

$$\begin{aligned} U_f(|x\rangle|-\rangle) &= U_f \frac{1}{\sqrt{2}} (|x\rangle|0\rangle - |x\rangle|1\rangle) \\ &= \frac{1}{\sqrt{2}} (U_f|x\rangle|0\rangle - U_f|x\rangle|1\rangle) \\ &= \frac{1}{\sqrt{2}} (|x\rangle|0 \oplus f(x)\rangle - |x\rangle|1 \oplus f(x)\rangle) \\ &= \frac{1}{\sqrt{2}} (|x\rangle|f(x)\rangle - |x\rangle|\overline{f(x)}\rangle) \end{aligned}$$

$$= \begin{cases} \frac{1}{\sqrt{2}}(|x\rangle|0\rangle - |x\rangle|1\rangle) & : \text{if } f(x) = 0 \\ \frac{1}{\sqrt{2}}(|x\rangle|1\rangle - |x\rangle|0\rangle) & : \text{if } f(x) = 1 \end{cases}$$

Therefore,

$$U_f(|x\rangle|-\rangle) = (-1)^{f(x)}|x\rangle|-\rangle$$

Here, instead of the target qubit, the control qubit is affected. This qubit will also be the one measured later.

This phase oracle will be used later for the derivation of Deutsch's algorithm.

3.2.2 Deutsch's Algorithm:

Before extending to n-bit inputs, Deutsch's algorithm helps to see the logic work on the simplest possible case — a single input qubit, and a function with only four possible behaviours

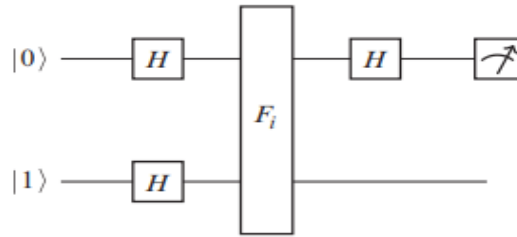


Figure 2: Quantum circuit for Deutsch's algorithm.

Step 1: Initialisation

Initialise with: $|0\rangle \otimes |1\rangle$

Step 2: Creation of Superposition

Apply the Hadamard gate on both qubits.

$$\begin{aligned} H(|0\rangle \otimes |1\rangle) &= \left(\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \right) \otimes \left(\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \right) \\ &= |+\rangle|-\rangle \\ &= \left(\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \right) |-\rangle \\ &= \frac{1}{\sqrt{2}}(|0\rangle|-\rangle + |1\rangle|-\rangle) \end{aligned}$$

Step 3: Oracle Application

Apply the oracle U_f

$$\begin{aligned}
&= U_f \left(\frac{1}{\sqrt{2}} (|0\rangle |-\rangle + |1\rangle |-\rangle) \right) \\
&= \frac{1}{\sqrt{2}} (U_f |0\rangle |-\rangle + U_f |1\rangle |-\rangle)
\end{aligned}$$

Applying phase oracle,

$$\begin{aligned}
&= \frac{1}{\sqrt{2}} ((-1)^{f(0)} |0\rangle |-\rangle + (-1)^{f(1)} |1\rangle |-\rangle) \\
&= \frac{1}{\sqrt{2}} ((-1)^{f(0)} |0\rangle + (-1)^{f(1)} |1\rangle) |-\rangle
\end{aligned}$$

Now we apply both possibilities of the function-constant and balanced, one by one.

If U_f is constant,

$$\begin{aligned}
&= \begin{cases} \frac{1}{\sqrt{2}} ((-1)^0 |0\rangle + (-1)^0 |1\rangle) |-\rangle & : \text{if } \begin{cases} f(0) = 0 \\ f(1) = 0 \end{cases} \\ \frac{1}{\sqrt{2}} ((-1)^1 |0\rangle + (-1)^1 |1\rangle) |-\rangle & : \text{if } \begin{cases} f(0) = 1 \\ f(1) = 1 \end{cases} \end{cases} \\
&= \begin{cases} \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) |-\rangle & : \text{if } \begin{cases} f(0) = 0 \\ f(1) = 0 \end{cases} \\ \frac{1}{\sqrt{2}} (-|0\rangle - |1\rangle) |-\rangle & : \text{if } \begin{cases} f(0) = 1 \\ f(1) = 1 \end{cases} \end{cases} \\
&= \begin{cases} |+\rangle |-\rangle & : \text{if } \begin{cases} f(0) = 0 \\ f(1) = 0 \end{cases} \\ -|+\rangle |-\rangle & : \text{if } \begin{cases} f(0) = 1 \\ f(1) = 1 \end{cases} \end{cases} \\
&= \pm |+\rangle |-\rangle
\end{aligned}$$

Therefore, if the oracle is constant, the control qubit collapses to the state $|+\rangle$ up to a global phase.

If U_f is balanced,

$$\begin{aligned}
&= \begin{cases} \frac{1}{\sqrt{2}} ((-1)^0 |0\rangle + (-1)^1 |1\rangle) |-\rangle & : \text{if } \begin{cases} f(0) = 0 \\ f(1) = 1 \end{cases} \\ \frac{1}{\sqrt{2}} ((-1)^1 |0\rangle + (-1)^0 |1\rangle) |-\rangle & : \text{if } \begin{cases} f(0) = 1 \\ f(1) = 0 \end{cases} \end{cases}
\end{aligned}$$

$$\begin{aligned}
&= \begin{cases} \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)|-\rangle & : \text{if } \begin{cases} f(0) = 0 \\ f(1) = 1 \end{cases} \\ \frac{1}{\sqrt{2}}(-|0\rangle + |1\rangle)|-\rangle & : \text{if } \begin{cases} f(0) = 1 \\ f(1) = 0 \end{cases} \end{cases} \\
&= \begin{cases} |-\rangle|-\rangle & : \text{if } \begin{cases} f(0) = 0 \\ f(1) = 1 \end{cases} \\ -|-\rangle|-\rangle & : \text{if } \begin{cases} f(0) = 1 \\ f(1) = 0 \end{cases} \end{cases} \\
&= \pm |-\rangle|-\rangle
\end{aligned}$$

Therefore, if the oracle is balanced, the control qubit collapses to the state $|-\rangle$ up to a global phase.

Step 4: Interference

After applying the oracle, Hadamard gates are applied again to the input qubits. This leads to interference between the amplitudes:

Constructive and destructive interference determine the final measurement outcome.

$$\begin{aligned}
\begin{cases} H(|+\rangle) = |0\rangle \\ H(|-\rangle) = |1\rangle \end{cases} & : \text{if } U_f \text{ is constant} \\
& : \text{if } U_f \text{ is balanced}
\end{aligned}$$

Step 5: Measurement

Measure the control qubit.

If the oracle is constant, the qubit collapses to the state of 0 with 100% chance.

If the oracle is balanced, the qubit collapses to the state of 1 with 100% chance.

The oracle encodes the function into the phase of the quantum state, and interference reveals the result in a single measurement.

This algorithm is primarily of theoretical importance, as it demonstrates a clear separation between classical and quantum computational capabilities. It was the first example of proving that there are quantum algorithms faster than classical ones.

This works cleanly for one input bit. The natural question is whether the same trick scales — and it does, almost elegantly.

3.2.3 Deutsch-Jozsa Algorithm:

The Deutsch–Jozsa algorithm takes the same core idea — superposition, phase kickback, interference — and stretches it across n input qubits. The problem grows exponentially harder for a classical computer, but the quantum circuit barely changes.

Given any function $f(x_0, x_1, \dots, x_{n-1})$ that has n boolean inputs and has just one boolean output, we construct the gate F given by the following circuit, where the slashes with n on the top lines indicate that we have n wires in parallel.

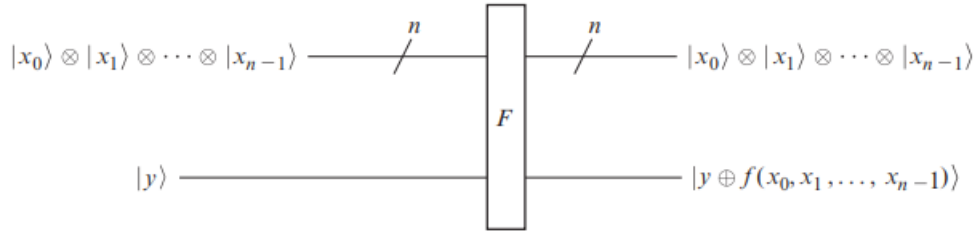


Figure 3: Reversible quantum circuit for n input bits.

The qubits are initialised as

$$|0\rangle^{\otimes n} \otimes |1\rangle$$

to get this circuit:

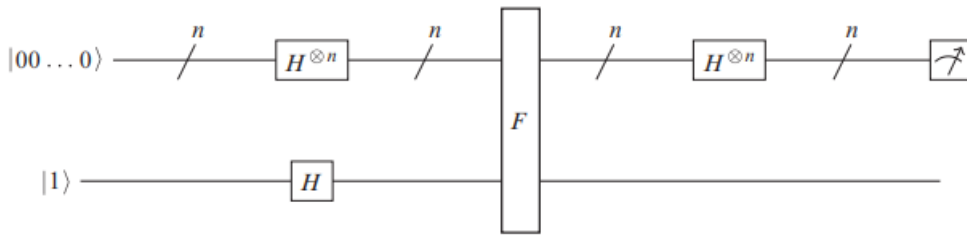


Figure 4: Quantum circuit for the Deutsch–Jozsa algorithm.

Step 1: Creation of Superposition

Apply the Hadamard gate on both qubits.

$$H^{\otimes n}|0\rangle^{\otimes n} \otimes H|1\rangle$$

If $n=2$:

$$\begin{aligned} & H^{\otimes 2}|0\rangle^{\otimes 2} \otimes |-\rangle \\ & \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)|-\rangle \\ & \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)|-\rangle \\ & = \frac{1}{\sqrt{2^2}} \sum_{x \in \{0,1\}^2} |x\rangle |-\rangle \end{aligned}$$

The case where $n = 2$ is shown to keep the notation and calculations less cluttered, although every step applies identically for any value of n .

Generalising this would give:

$$H^{\otimes n}|0\rangle^{\otimes n} \otimes H|1\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |-\rangle$$

Step 2: Oracle Application

Apply the oracle U_f

$$\begin{aligned} U_f(H^{\otimes n}|0\rangle^{\otimes n} \otimes |-\rangle) &= U_f \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |-\rangle \\ &= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} U_f |x\rangle \otimes |-\rangle \end{aligned}$$

Applying phase oracle,

$$= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle \otimes |-\rangle$$

Step 3: Application of $H^{\otimes n}$ to control the qubit

$$= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} H^{\otimes n} |x\rangle \otimes |-\rangle$$

Finding $H^{\otimes n}|x\rangle$:

If $n=2$:

$$|x\rangle = |x_1 x_2\rangle$$

$$H^{\otimes n}|x_1 x_2\rangle = (H|x_1\rangle) \otimes (H|x_2\rangle)$$

$$= \left(\frac{1}{\sqrt{2}} (|0\rangle + (-1)^{x_1} |1\rangle) \right) \otimes \left(\frac{1}{\sqrt{2}} (|0\rangle + (-1)^{x_2} |1\rangle) \right)$$

$$= \frac{1}{2} ((|0\rangle + (-1)^{x_1} |1\rangle) \otimes (|0\rangle + (-1)^{x_2} |1\rangle))$$

$$= \frac{1}{2} (|00\rangle + (-1)^{x_2} |01\rangle + (-1)^{x_1} |10\rangle + (-1)^{x_1+x_2} |11\rangle)$$

A pattern is found that the (-1) factor is found only when the respective qubit contains a 1.

Therefore, this can be rewritten as,

$$\frac{1}{2} ((-1)^{x_1(0)+x_2(0)} |00\rangle + (-1)^{x_1(0)+x_2(1)} |01\rangle + (-1)^{x_1(1)+x_2(0)} |10\rangle + (-1)^{x_1(1)+x_2(1)} |11\rangle)$$

Therefore, generalising to n gives:

$$H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} (-1)^{x \cdot z} |z\rangle$$

where z is all the possible outcome vectors.

Applying this in " $\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} H^{\otimes n}|x\rangle \otimes |-\rangle$ "

$$\begin{aligned} \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} H^{\otimes n}|x\rangle \otimes |-\rangle &= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} (-1)^{x \cdot z} |z\rangle \otimes |-\rangle \\ &= \frac{1}{2^n} \sum_{x \in \{0,1\}^n} \sum_{z \in \{0,1\}^n} (-1)^{f(x) + x \cdot z} |z\rangle \otimes |-\rangle \end{aligned}$$

For $|z\rangle = |0\rangle^{\otimes n}$.i.e., $|00 \dots 0\rangle$:

$$= \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x) + 0} |00 \dots 0\rangle \otimes |-\rangle$$

If U_f is constant,

$$= \begin{cases} \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^0 |00 \dots 0\rangle \otimes |-\rangle & : \text{if } \begin{cases} f(0) = 0 \\ f(1) = 0 \end{cases} \\ \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^1 |00 \dots 0\rangle \otimes |-\rangle & : \text{if } \begin{cases} f(0) = 1 \\ f(1) = 1 \end{cases} \end{cases}$$

$$= \begin{cases} \frac{1}{2^n} \overbrace{(1 + 1 + \dots + 1)}^{2^n \text{ times}} |00 \dots 0\rangle \otimes |-\rangle & : \text{if } \begin{cases} f(0) = 0 \\ f(1) = 0 \end{cases} \\ \frac{1}{2^n} \overbrace{(-1 - 1 - \dots - 1)}^{2^n \text{ times}} |00 \dots 0\rangle \otimes |-\rangle & : \text{if } \begin{cases} f(0) = 1 \\ f(1) = 1 \end{cases} \end{cases}$$

$$= \frac{1}{2^n} (\pm 2^n) |00 \dots 0\rangle \otimes |-\rangle$$

$$= 1 |00 \dots 0\rangle \otimes |-\rangle$$

i.e, chance of measuring $00 \dots 0$ is 100%

If U_f is balanced,

Half of the outputs must be 0, and the other half must be 1 since the oracle is a balanced function.

$$= \frac{1}{2^n} (-1 - 1 \dots + 1 + 1) |00 \dots 0\rangle \otimes |-\rangle$$

Here, all -1s and +1s cancel out to give 0

$$= 0 |00 \dots 0\rangle \otimes |-\rangle$$

i.e, chance of measuring $00\dots 0$ is 0%.

Therefore, the qubit will collapse to any other qubit other than $00\dots 0$

Step 4: Measurement

Measure the control qubit.

- If the oracle is constant, then the qubit will collapse to the state of $00\dots 0$ with 100% chance.
- If the oracle is balanced, then the qubit will collapse to the state of $00\dots 0$ with 0% chance. Instead, it will collapse to any other state, such as $|10\dots 01\rangle$, $|10\dots 0\rangle$, etc.

This result shows that interference amplifies the correct outcome while cancelling incorrect states.

This exponential blowup is exactly the wall the quantum approach walks straight through.

4. The *Qiskit* Implementation:

4.1 Introduction

The Deutsch–Jozsa algorithm was implemented using Qiskit, IBM’s open-source quantum computing framework. The circuit was simulated using a quantum simulator to verify the algorithm's theoretical predictions.

However, unlike a real quantum computer, simulations run in an ideal environment without hardware noise or physical imperfections.

Here, we use a fake backend -the *Qiskit Aer simulator framework* to recreate the noise like a real quantum computer.

4.2 Application

To study the effect of realistic quantum errors, a noisy simulation environment was introduced using Qiskit's noise modelling tools. A fake backend was used to simulate imperfections such as gate noise and measurement errors.

We initialise two qubits with states of $|0\rangle$ for both.

```
qc = QuantumCircuit(3, 2)
```

Prepare the output qubit in the $|-\rangle$ state so the oracle can encode information into phase.

```
qc.x(2)
```

```
qc.h(2)
```

Hadamard gates are applied to the input qubits to create a superposition of all possible input states, allowing the oracle to evaluate multiple inputs simultaneously.

```
qc.h(0)
```

```
qc.h(1)
```

Controlled-X gates were used to implement the balanced oracle. These operations encode the function into the phase of the quantum state by conditionally flipping the output qubit based on the input qubits.

```
qc.cx(0, 2)
```

```
qc.cx(1, 2)
```

Another layer of Hadamard gates is applied to the input qubits after the oracle. This causes the quantum states to interfere with each other, cancelling some outcomes while strengthening others, which makes it possible to identify whether the function is constant or balanced after measurement.

```
qc.h(0)
```

```
qc.h(1)
```

Finally, the input qubits are measured. The measurement result determines whether the function is constant or balanced.

```
qc.measure(0, 0)
```

```
qc.measure(1, 1)
```

Running this on a *noisy simulator* — rather than an ideal one — gives a more honest picture of what the algorithm looks like in practice.

```
simulator = Aer.get_backend('aer_simulator')
```

```
fake_backend = GenericBackendV2(num_qubits=5)
```

```
noise_model = NoiseModel.from_backend(fake_backend)
```

Due to the noise, the dominant state or the state that appeared the most is taken into consideration. Using this, it can be determined whether the function is constant or not.

```
dominant_state = max(counts, key=counts.get)
if dominant_state == '00':
    print("Constant")
else:
    print("Balanced")
```

5. Results

The following table summarises the major differences between the classical and quantum approaches used in this study.

Feature	Classical Approach	Quantum Approach (Deutsch–Jozsa)
Computational Model	Classical computation	Quantum computation
Information Unit	Bit	Qubit
State Representation	0 or 1	Superposition of 0 and 1
Number of Queries	Up to $2n^2 + 1$	1
Method Used	Repeated function evaluation	Superposition and interference
Parallelism	Sequential evaluation	Quantum parallelism
Efficiency for Large n	Exponentially inefficient	Significantly more efficient
Key Operations	Classical logic operations	Hadamard gates, oracle operations, interference
Output Determination	Multiple evaluations required	Single measurement sufficient
Practical Significance	Conventional computation	Demonstrates quantum advantage

Table 1: Comparison Between Classical and Quantum Approaches

The Deutsch–Jozsa circuit was executed using the Qiskit simulator, and the measurement results were visualised using a histogram.

Output:

Balanced

{'11': 1002, '01': 16, '10': 6}

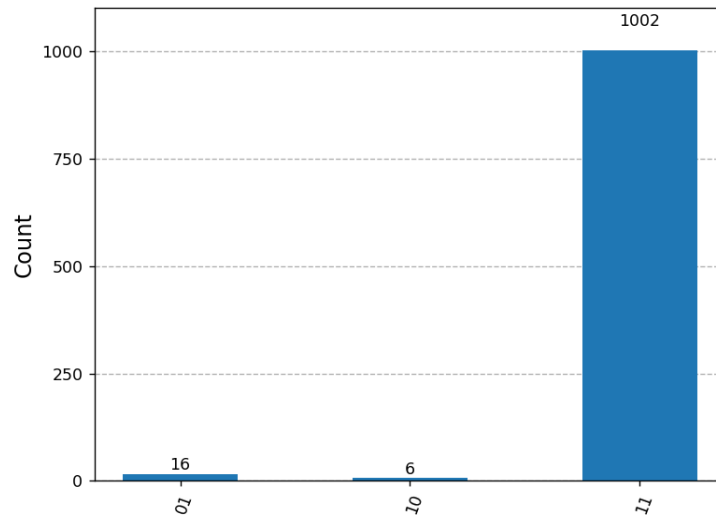


Figure 5: Histogram showing measurement outcomes for the balanced oracle.

The result shows that 1002 shots have produced 11, whereas 22 shots have yielded a deviation. Thus, the dominant outcome is taken into consideration.

The observed output state was $|11\rangle$ with the highest probability. According to the Deutsch–Jozsa algorithm, obtaining the state other than $|00\dots 0\rangle$ indicates that the oracle represents a balanced function.

Small variations or noise in the output arise from simulator approximations, gate imperfections, or quantum decoherence in *real quantum hardware*. These effects can slightly alter the measurement probabilities, causing outputs other than the expected state to appear with low frequency.

Even under simulated noise, the algorithm identified the balanced oracle correctly in 98.85% of shots — the small deviation tells you more about real quantum hardware than any perfect simulation could.

6. Conclusion:

The mathematical derivation, circuit analysis, and Qiskit implementation demonstrated how quantum computation can determine whether a function is constant or balanced using significantly fewer queries than classical methods.

The comparison done here between the classical and quantum approaches highlights the computational advantage of quantum algorithms. While the classical method might require up

to $2^{n-1} + 1$ queries to determine whether a function is constant or balanced, the Deutsch–Jozsa algorithm achieves the same result using only a single oracle query. This improvement is made possible through the use of superposition, phase kickback, and quantum interference.

The results didn't just confirm the implementation — they showed that the algorithm is robust enough to survive realistic noise, which is the actual barrier between quantum theory and quantum hardware.

Instead of checking inputs one by one like a classical computer, a quantum computer can encode information into quantum phases and use interference to reveal the answer efficiently. This same idea later became the foundation of important algorithms like Shor's algorithm and Grover's algorithm.

7. Future Scope:

The Deutsch–Jozsa algorithm cannot break passwords or simulate medicines. By itself, it solves a problem with little practical use. However, its importance lies elsewhere. It was the first algorithm to mathematically prove that quantum computers are fundamentally different from classical computers, not just faster versions of them.

Every time a quantum algorithm outruns its classical counterpart, it's borrowing the same intuition Deutsch put on paper in 1985.

The single query isn't the result. The single query is the idea.

The core ideas — superposition, phase encoding, and interference — aren't limited to this algorithm alone. They show up at the base of Shor's and Grover's algorithms too, making Deutsch–Jozsa a useful conceptual stepping stone rather than an isolated curiosity.

The interesting part is that such things don't stop at simulators; real quantum hardware provides scope for more improvement and opens a whole new ground. The noise, decoherence, and gate errors that come with physical hardware aren't just obstacles; they're problems that push the field forward, driving better error correction, more resilient algorithm design, and hardware that improves with each failure.

Quantum computing still has many challenges to overcome and much more to achieve. Its true importance lies not only in what it has already accomplished, but also in the possibilities it may unlock in the future.

8. Acknowledgement

I would like to express my sincere gratitude to my school and the mentoring professors for providing me with the opportunity and guidance to explore quantum computing through this research project.

I am especially thankful to Chris Bernhardt, whose book *Quantum Computing for Everyone* greatly helped me build an intuitive understanding of quantum computing concepts. His explanations played an important role in shaping my understanding of the Deutsch and

Deutsch–Jozsa algorithms. I am also grateful that he took the time to respond to my email, which further encouraged my interest in the subject.

I would also like to thank the developers and contributors of Qiskit and IBM for providing open-source tools and educational resources that made the practical implementation of this project possible.

Finally, I thank my teachers, family, and everyone who supported and motivated me throughout this work.

9. References:

- [1] Qiskit, "Qiskit Documentation," IBM. [Online]. Available: <https://docs.quantum.ibm.com/> [Accessed: 11-May-2026].
- [2] D. Deutsch and R. Jozsa, "Rapid solution of problems by quantum computation," *Proc. R. Soc. Lond. A, Math. Phys. Sci.*, vol. 439, no. 1907, pp. 553–558, Dec. 1992.
- [3] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge, U.K.: Cambridge University Press, 2000 [Accessed: 11-May-2026].
- [4] IBM Quantum, "IBM Quantum Platform," IBM. [Online]. Available: <https://quantum.ibm.com/> [Accessed: 11-May-2026].
- [5] Quantum Soar, "Deutsch-Jozsa Algorithm: Exponential Speedup on a Quantum Computer!" *YouTube*, Mar. 5, 2023. [Online Video]. Available: <https://youtu.be/jfJckA7Amik> [Accessed: 11-May-2026].
- [6] C. Bernhardt, *Quantum Computing for Everyone*. Cambridge, MA: MIT Press, 2019.